Minor Project(IT897)

Layout-Aware Hierarchical Representation for RAG

BY
Mr. KSHITIJ KHEDE (252IT010)
Department of Information Technology

National Institute of Technology Karnataka, Surathkal-575025

Under the Guidance of
Prof. ANANTHANARAYANA V. S.

Table of Contents

- Introduction
- Literature Survey
- Problem Statement and Objectives
- Methodology Used in this Work
- Architecture
- Results and Analysis
- Discussion
- Conclusion and Future Work
- References

Introduction

- Retrieval-Augmented Generation (RAG) lets an LLM answer using external documents instead of only its training data.
- The quality of a RAG answer depends heavily on how the source document is represented before retrieval.
- Research papers and technical PDFs are highly structured multi-column layouts, numbered sections, and a title → section → subsection hierarchy.
- Standard RAG pipelines flatten a PDF into plain text and fixed-size chunks, which destroys this structure:
  - -multi-column pages are read in the wrong order (text jumps across columns),
  - -the parent–child relationship between sections is lost.
- This work (minor project) builds a layout-aware, hierarchical knowledge representation — SHIA-RAG — as the foundation on which reliable retrieval can later be built.

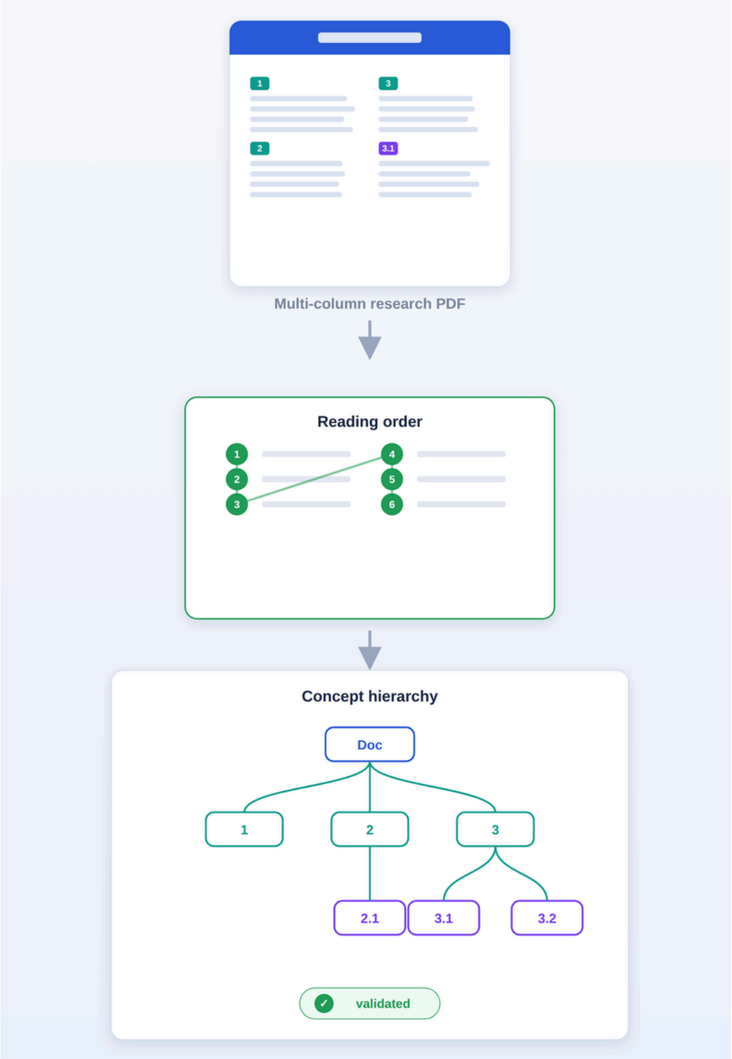


Fig.1.Structured document to a validated concept hierarchy

Literature Survey

Literature Survey — Structure-Aware Retrieval

- Retrieval-Augmented Generation (RAG) improves an LLM's answers by retrieving relevant content from external documents at query time.
- The quality of a RAG answer depends heavily on how a document is represented and chunked before retrieval — poor chunking loses information and breaks the links between related parts.
- Early RAG systems split a document into flat, fixed-size passages, which discards its natural structure and reading order.
- Recent research therefore moves toward hierarchical, structure-aware representations of documents.
- Three works are most closely related to my project, each building document hierarchy in a different way:
  - **RAPTOR** — a tree built by recursive semantic clustering + summarisation
  - **TreeRAG** — tree-chunking that stores and retrieves content along the document's hierarchy
  - **HiChunk** — multi-level chunking using fine-tuned LLMs, with a dedicated evaluation benchmark
- These are reviewed next, followed by the gap they leave open.

Literature Survey Contd.

RAPTOR — Recursive Tree-Organized Retrieval

Key points:

- **Problem addressed**: standard RAG retrieves only short contiguous chunks, limiting holistic understanding of a long document.
- **Core idea**: recursively embed → cluster → summarise chunks, building a tree bottom-up with different levels of summarisation.
- At query time, retrieves from this tree, combining information at multiple levels of abstraction.
- **Result**: significant gains on multi-step QA; e.g. +20% absolute on the Quality benchmark with GPT-4.
- **Open issue**: the tree is built from semantic similarity (clustering) — it does not use the document's own layout or native section hierarchy.

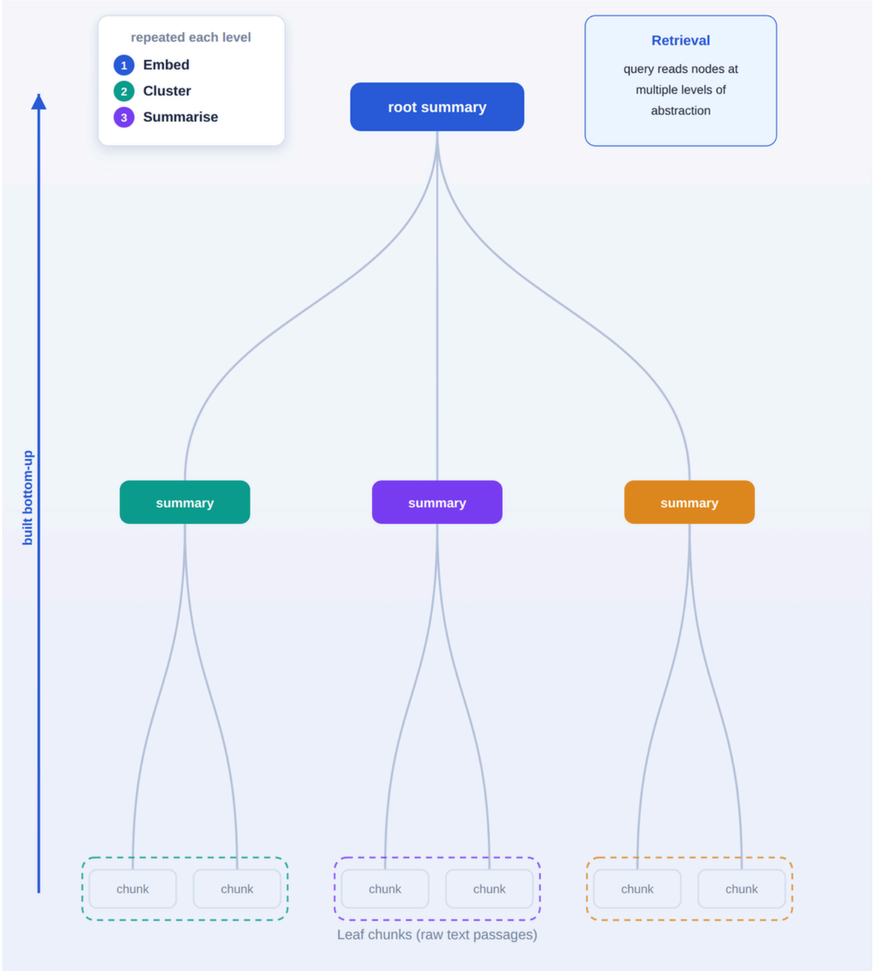


Fig.2. RAPTOR's recursive tree-building process

Literature Survey Contd.

TreeRAG — Hierarchical Storage for Long Documents

Key points:

- **Problem addressed:** naive chunking destroys chunk integrity and breaks the connections between related chunks, hurting Query-Focused Summarisation.
- **Core idea:** Tree-Chunking — chunk and embed content in a tree-like structure that preserves the document's hierarchy.
- **Retrieval:** Bidirectional Traversal Retrieval — "root-to-leaves" and "leaf-to-roots" — to keep related chunks connected and reduce noise.
- **Evaluated on:** Finance, Law, and Medical subsets of the Dragonball dataset; improves recall quality and precision.
- **Open issue:** relies on the tree structure but does not address multi-column reading-order recovery or formal validation of the built tree.

Literature Survey Contd.

HiChunk — Hierarchical Chunking

Key points (HiChunk):

- **Problem addressed:** document chunking lacks good evaluation; existing benchmarks miss chunking quality due to "evidence sparsity."
- **Core idea:** HiChunk framework — multi-level (hierarchical) document structuring using fine-tuned LLMs, plus an Auto-Merge retrieval algorithm.
- Introduces HiCBench, a benchmark with manually annotated multi-level chunking points.

Outcome of the survey:

- Existing methods build hierarchy from semantics or learned models — the document's native layout and section numbering are underused.
- Multi-column reading order is rarely recovered explicitly before structuring.
- The induced hierarchy is not formally validated (no cycle / single-parent / depth guarantees).

Problem Statement and Objectives

Problem Statement:

To transform a layout-rich document into a correct reading order and a validated hierarchical concept representation that preserves its native structure for reliable retrieval.

Objectives:

- Recover reading order — reconstruct the correct reading sequence from multi-column page layouts.
- Induce the concept hierarchy — build the title → section → subsection tree from the document's native section numbering, with a semantic fallback for unnumbered headings.
- Validate the structure — guarantee the hierarchy is acyclic, single-parent, and depth-bounded, with no orphan concepts, and assign a confidence score to each node.
- Evaluate on real documents — measure reading-order accuracy and parent-assignment accuracy on a corpus of research papers.

Methodology Used in this Work

Methodology — Column-Gutter Reading-Order Recovery

- Problem: naive top-to-bottom sorting jumps between columns on multi-column pages.
- Idea: detect the column gutter and read each column fully before moving on.
  - Steps:Split the page at the vertical mid-line (gutter at page-width / 2)
  - Classify each block: left column, right column, or spanning (width > 0.60 × page width)
  - Place spanning blocks into top or bottom bands (title, full-width figures)
  - Within a column, cluster blocks into lines using a vertical band tolerance τ_y = 20 pt, then order top-to-bottom
- Output order: top-spanning → left column → right column → bottom-spanning

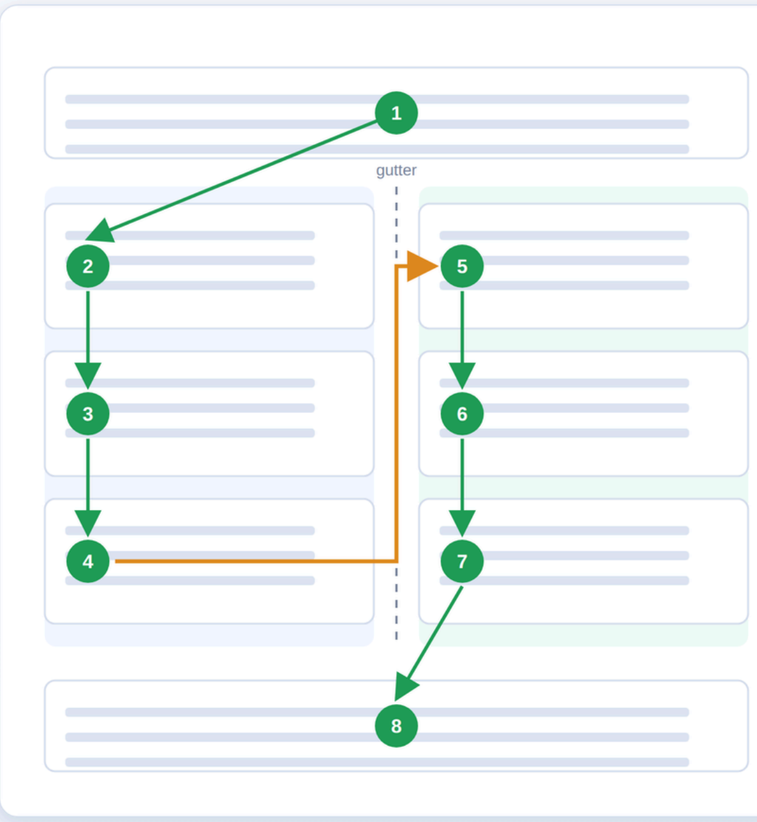


Fig. 3. Column-gutter reading-order

Methodology Used in this Work Cond.

Methodology — Hierarchy, Validation and Confidence

Hierarchy induction (numbering-primary, semantic fallback):

- Primary signal: section numbering — "3.1" attaches to "3" (deterministic, exact)
- Fallback — Parent Ranking Engine (PRE) for unnumbered headings: Score(u,c) = 0.40·S_struct + 0.40·S_sem + 0.20·S_level
  - S_struct = 1 / (1 + log(1 + |order(c) − order(u)|))
  - S_sem = TF-IDF cosine similarity (TF-IDF)
  - S_level = max(0, 1 − 0.2·(Δlevel − 1))

Graph validation (guarantees, before each edge is added):

- No cycles (upward-DFS), single parent per node, depth ≤ D_max = 8, document isolation → no orphans

Confidence — KCE (topological propagation):

- C(v) = C_local(v) · (δ + (1 − δ)·C(parent)), with δ = 0.85
- Propagated in topological order (Kahn's algorithm)

Architecture

System Architecture

- Dual-tier data model (validated schemas): Tier 1 = physical Text Blocks, Tier 2 = Concept Nodes.
  a. Six modules, arranged as a pipeline:PDF Ingestion — extract text blocks with geometry + font (pdfplumber)
  b. Structural Parser — column-gutter reading order + heading classification
  c. Concept Extractor — headings → concept nodes, grounded to blocks
  d. Hierarchy Builder — numbering-primary attachment, PRE fallback
  e. Graph Validator — acyclic, single-parent, depth-bound checks
  f. KCE Scorer — topological confidence propagation
- Data flow: PDF → blocks → ordered blocks → concepts → validated hierarchy → confidence-scored tree.

Architecture

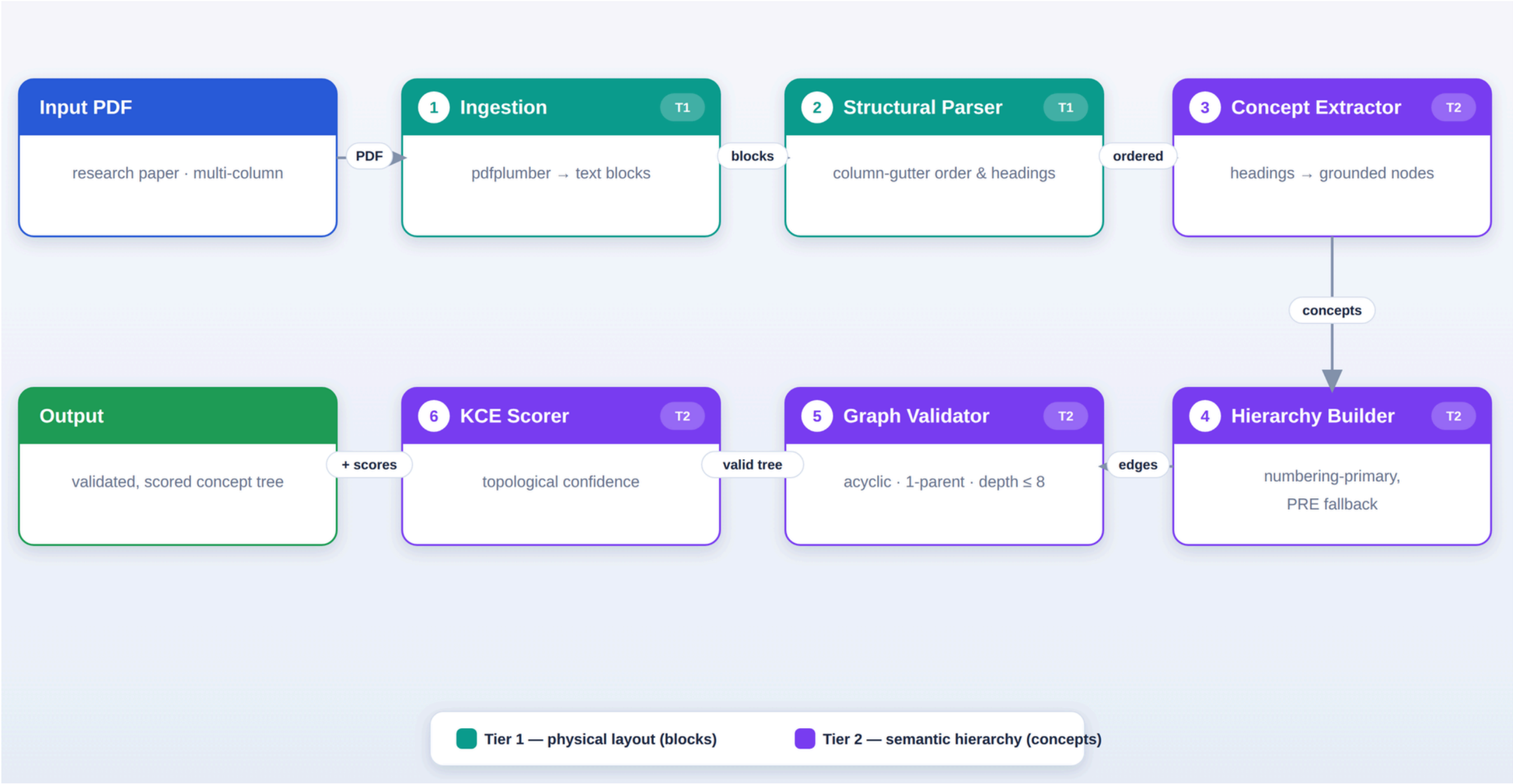


Fig.4. Complete system architecture: a six-module pipeline over a dual-tier data model.

Results and Analysis

Results — Reading Order & Structure

Metric: Inversion rate = fraction of block pairs read out of order vs. a column-first reference;
ROM (reading-order monotonicity) = 100% − inversion rate. Lower inversion / higher ROM is better.

Reading-order recovery (averaged over all 11 papers):

Table 1: Reading-order recovery (naive vs. column-gutter).

Method	Inversion ↓	Monotonicity (ROM) ↑
Naive top-to-bottom	21.8%	78.2%
Column-gutter (ours)	0.5%	99.5%

- ≈ 44× fewer reading-order inversions than naive extraction.
- Naive extraction fails mainly on multi-column pages (text jumps across the gutter);
  the gutter algorithm reads each column fully, in order.

Structural soundness (Graph Validator, checked on every edge, all 11 documents):

- 0 cycles — verified by upward-DFS (acyclic)
- Single parent per concept — no ambiguous attachments
- Depth ≤ 8 (D_max bound) on every document
- 0 orphans — document isolation keeps every node connected to its root

Results and Analysis Contd.

Results: Parent-Assignment Accuracy

Parent-Assignment Accuracy (PAA) over 196 numbered sections (11 papers), vs. numbering-derived gold:

Table 2: Parent-Assignment Accuracy of attachment strategies.

Strategy	PAA
Sequential	17.9%
Lexical overlap	34.7%
PRE (semantic)	51.5%
Numbering (primary)	100.0%

Finding: numbering is exact for numbered papers; PRE is the strongest content-based strategy and serves as the fallback for unnumbered headings.

Discussion

- Fixing reading order first (21.8% → 0.5%) is the foundation — every later stage depends on correct block order.
- Almost all the gain comes from correctly handling multi-column pages.
- Numbering is exact (100%) — a section number encodes its own parent (3.1 → 3), so no inference is needed.
- Among content-based strategies, PRE (51.5%) beats lexical (34.7%) and sequential (17.9%) — the right fallback when numbering is absent.
- Validation by construction — every edge checked before insertion, so 0 cycles, 0 orphans, depth ≤ 8, with no cleanup step.
- Case study (TreeRAG): 16 pages → 15 concepts, 1 root, depth 3, in ≈ 2 s.
- Limitation: semantic score is TF-IDF, not a dense encoder → neural PRE in future work.
- Limitation: works on text-based PDFs; reading-order reference is geometric (no human-annotated gold).

Conclusion and Future Work

Conclusion:

- Built a layout-aware, hierarchical, validated document representation for RAG (the minor-project problem).
- Column-gutter reading-order recovery cut inversions from 21.8% → 0.5% (ROM 99.5%).
- Numbering-primary + PRE-fallback hierarchy — 100% exact on numbered sections; PRE the best content-based fallback.
- Graph validation guarantees soundness — 0 cycles, 0 orphans, depth ≤ 8 across all 11 papers.
- Delivered as a reproducible six-module pipeline with tests.

Future Work:

- Replace TF-IDF with a dense neural encoder (e.g. sentence embeddings) for stronger PRE.
- Add cross-document linking to connect concepts across papers.
- Build retrieval + answer generation on top of the validated hierarchy (full RAG).
- Extend to scanned PDFs via OCR, and evaluate against human-annotated gold.
- This minor project becomes one objective of the major project.

References

[1] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," in Proc. Int. Conf. on Learning Representations (ICLR), 2024.

[2] W. Tao, X. Xing, Y. Chen, L. Huang, and X. Xu, "TreeRAG: Unleashing the Power of Hierarchical Storage for Enhanced Knowledge Retrieval in Long Documents," South China University of Technology.

[3] W. Lu, K. Chen, R. Qiao, and X. Sun, "HiChunk: Evaluating and Enhancing Retrieval-Augmented Generation with Hierarchical Chunking," Tencent Youtu Lab, 2025.

THANK YOU